Contents

Chapter 1: Title & Introduction of Project 1

1.1 Title 1

1.2 Introduction 1

Chapter 2: Scope, Objective and Problem Definition 1

2.1 Scope 1

2.2 Objective 2

2.3 Problem Definition 2

Chapter 3: Project Category and Implementation Methodology 3

3.1 Project Category 3

3.2 Implementation Methodology 3

Chapter 4: Module Description and Process Logic 4

4.1 Module Description 4

4.2 Process Logic 5

Chapter 5: Analysis 9

5.1 Use Case Diagram 9

5.2 Class Diagram 10

5.3 Data Flow Diagram 11

5.4 Entity Relationship Diagram 15

5.5 Data Structure 18

Chapter 6: System Requirement Specification and Feasibility Study 19

6.1 Software 19

6.2 Hardware 22

6.3 Feasibility Study 22

Chapter 7: System Development Life Cycle 23

Chapter 8: Testing and Security 23

8.1 Testing of the Project 23

8.2 Security Implementation 24

Chapter 9: Industry Client Interface and Future Scope 25

9.1 Industry Client Interface 25

9.2 Future Scope and Further Enhancement 25

References 26

CHAPTER 1: TITLE & INTRODUCTION OF PROJECT

1.1 Title

HNAZER

Lifestyle Journal and Self-Analysis Tracker

Item	Details
Project title	Hnazer – Lifestyle Journal and Self-Analysis Tracker
Project category	Web application; Data management (browser object store); Information security (local accounts, salted password hashing, role-based screens); Applied AI (rule-based analysis with an optional Gemini advisor)
Front end	React 18, Vite 6, Tailwind CSS 3 (with PostCSS and Autoprefixer), Lucide icons, canvas-confetti; charts, radar and heatmap are drawn as inline SVG
Back end	None. There is no application server: all logic runs in the browser. Vite is used only as the development server and build tool.
Database	IndexedDB (database hnazer-browser-data, object store accounts with an auto-increment key and a unique username index); a small session pointer is kept in localStorage
AI service	Built-in rule-based advisor (runs in the browser); optional Google Gemini API, used only on explicit choice
Output	Dashboard cards, trend charts, radar and heatmap, burnout-risk summary, history table, CSV export, JSON backup, printable report, administrator console

1.2 Introduction

Many people want to understand how their daily habits affect how they feel, but the relevant information is scattered. Sleep hours are noted in one place and workouts in another, while water intake and screen time are rarely recorded at all, and mood and diet are only remembered vaguely. Without a single record it is hard to notice patterns, such as short sleep combined with long screen time, or to see a slide towards burnout before it happens. Paper notebooks and spreadsheets can store such entries, but they need manual calculation and give no instant feedback. Many commercial health and fitness applications, on the other hand, depend on wearable sensors, ask for e-mail accounts or cloud sign-in, and keep personal data on third-party servers.

Hnazer is a local-first lifestyle journaling and self-analysis web application. A user creates a local account (username, name and password; no e-mail address is needed) and then records one daily entry covering sleep, exercise, water intake, screen time, mood, diet and optional notes. The system stores the account, profile and logs in the browser's own IndexedDB database, calculates a rule-based score for every metric, combines them into a weighted wellness score, and estimates a burnout risk from recent records with alerts and recommendations. The dashboard presents metric cards, trend charts, a mood distribution, a six-axis radar and a 35-day heatmap. A built-in advisor works entirely in the browser, while an optional Gemini advisor can be chosen explicitly for an external AI analysis. JSON and CSV export, a printable report, a reference library, a light/dark theme and editable daily targets complete the system. A separate administrator role, local to the same browser profile, can review accounts, inspect records, reset or delete accounts and take a full backup.

Hnazer has no server component, does not connect to biometric sensors and does not give medical diagnoses. All values are entered by the user, and every score, label and recommendation is an informational heuristic.

CHAPTER 2: SCOPE, OBJECTIVE AND PROBLEM DEFINITION

2.1 Scope

Within scope: local account management; daily journaling; rule-based scoring and burnout analysis; dashboard, charts and history; built-in and optional Gemini advisor; reference library; JSON and CSV export, JSON restore and printable report; daily targets, theme and log purge; browser-local administrator console.

Outside scope: biometric sensors or wearable integration; medically validated diagnosis or treatment advice; e-mail verification, password recovery and a user-facing account-deletion screen; multi-device synchronisation, cloud hosting or a shared server database; a central administrator view across devices; CSV import; a native mobile

application.

Intended users

User	Description	Access
Visitor	A person who opens the application without signing in.	Account screen only (sign in or create account).
Registered user	A person who journals and reviews his or her own habits.	Dashboard, daily entry, history, advisor, reference library, settings, backup and report; own data only.
Administrator	A browser-local role, seeded as the account "admin". It exists only in the browser profile where it was created.	Admin home and Admin Console: account list, statistics, account dossier, reset logs, delete account, full backup, change admin password. No daily entry or personal dashboard.
Developer / owner	The person who builds and deploys the static site.	Runs npm run dev / build. Cannot see visitors' data, which stays in each visitor's own browser.

2.2 Objective

- To provide local account registration, sign-in and sign-out, with validation errors shown clearly and each account's profile and logs kept separate.
- To record one daily entry per date (sleep, exercise, water, screen time, mood, diet, notes) that can be edited, deleted or topped up quickly with 250 ml of water.
- To calculate per-metric scores (0–100), a weighted wellness score and a burnout-risk level (Low, Moderate, Critical) with alerts and recommendations, using transparent rules.
- To present the data through metric cards, 7-day, 14-day, 30-day and all-record trend charts, a mood distribution, a six-axis radar and a 35-day heatmap.
- To offer a built-in advisor that sends nothing anywhere, and an optional, explicitly chosen Gemini advisor for external AI analysis.
- To provide search, filter, CSV export, JSON backup and restore, and a printable report.
- To let the user set personal daily targets and a screen-time ceiling, choose a light or dark theme, and clear the logs when required.
- To store data in the browser's IndexedDB with salted PBKDF2 password hashes, with no e-mail, server or cloud dependency.
- To provide a browser-local administrator console for reviewing, resetting and deleting accounts and for taking a full backup.
- To follow a maintainable structure (components, utility modules, one account-storage module) that can be extended later.

2.3 Problem Definition

Problem statement

Individuals who try to improve their routine find it hard to combine scattered habit data into one clear picture, and they rarely get timely feedback on risks such as too little sleep, excessive screen exposure or growing burnout. Existing tools are either too simple (plain notes with no analysis) or too demanding (sensors, cloud accounts, advertising, unclear data handling). The proposed system provides a lightweight, private journal that stores daily records per user in the browser and converts them into understandable scores, trends and advice. It supports self-awareness; it does not give medical advice or diagnosis.

Existing system and its limitations

- Paper journals and notes need manual review, give no score or trend, and are easy to lose.
- Spreadsheets need hand-written formulas, give no instant feedback and are not protected by a login.
- Many health and fitness apps depend on wearables, cloud accounts or e-mail sign-up, show advertisements and keep personal data on third-party servers.
- Few simple tools combine sleep, activity, hydration, screen time, mood and diet into one understandable score.
- Where AI features exist, users often cannot control what data is sent; a built-in option that needs no external service is rare.

- Small personal tools often lack backup, export or a printable summary.

Proposed system

Hnazer stores one profile and one logbook per account in the browser's database and calculates all insights from the stored data. The main advantages are:

- **Local-first:** accounts, profile and logs are kept in IndexedDB in the user's own browser profile; no e-mail address and no server are needed.
- **One daily entry** captures six habit areas; saving a date again replaces that date's entry, so every day has at most one record.
- **Transparent scoring:** fixed, documented rules give metric scores, a weighted wellness score and a burnout-risk level that the user can inspect.
- **Personal targets** (sleep, water, exercise and a screen-time ceiling) can be changed in Settings and feed the water, exercise and screen scores.
- **Optional AI:** the built-in advisor needs no network call; Gemini is used only when the user explicitly chooses it and supplies an API key.
- **Automatic saving:** every change to the profile or logs is written to IndexedDB through a serial save queue.
- **Portable data:** backup, restore, CSV export and a browser-printable report keep the data usable elsewhere.
- **Administration:** a browser-local administrator can review and maintain the accounts stored in the same browser profile.

CHAPTER 3: PROJECT CATEGORY AND IMPLEMENTATION METHODOLOGY

3.1 Project Category

The project belongs to the following categories of the BCSP-165 guidelines. It is a single-page web application: a React front end keeps its data in the browser's IndexedDB database and performs all calculations on the user's device.

Category	How it applies to this project
Data management	Account records (including profile and logs) are kept in an IndexedDB object store with an auto-increment primary key, a unique index on username, atomic read/write transactions, constraint errors mapped to user messages and serialised (queued) writes.
Web application / OOPS concepts	React components and state; separate utility modules for scoring, storage, account access and AI; objects model accounts, profiles and daily logs; a role field separates users from the administrator.
Information security	Salted PBKDF2-SHA-256 password hashes (210,000 iterations, 256-bit output), random 256-bit session tokens from the Web Crypto API, uniform error message for failed sign-in, input validation, role check for every administrator function, no plain passwords stored.
Information management / applied AI	Structured storage and reporting of personal habit data: scores, trends, risk summaries, CSV and JSON exports and a printable report; a deterministic rule-based advisor plus an optional external language-model advisor.

3.2 Implementation Methodology

Hnazer was built with an iterative, component-based approach. The interface is divided into React components, the business rules into small utility modules (scoring.js, aiService.js, storage.js) and all account storage into one module (localAccount.js). Each module was built, tried in the browser with sample data and refined before the next one was added, and the production build (npm run build) acts as the compile check. All logic runs on the client: Vite builds the application into a static site that any static host can serve, so no server has to be installed or maintained.

The architecture that results from this approach is described below.

Hnazer is a client-only application. The React interface is served as static files (by Vite during development, or from the static build in production). It calls the localAccount.js module, which uses IndexedDB for account records and localStorage for the session pointer, and the Web Crypto API for password hashing and random tokens. Scoring, charts, themes, exports and the built-in advisor run entirely in the browser. When the user chooses Gemini, the browser sends the prompt directly to Google's API. The page also requests font stylesheets from Google Fonts.

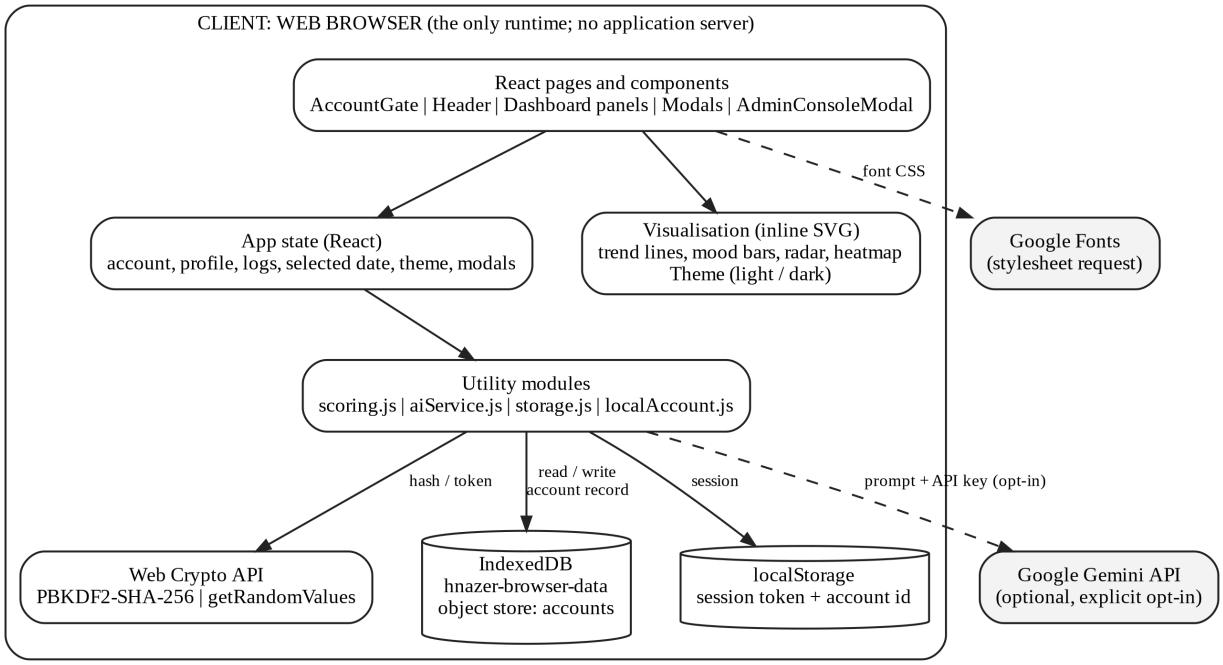


Figure 3.1: Architecture of Hnazer

Layer	Components	Responsibility
Presentation	React components (AccountGate, Header, VitalityOverview, MetricCards, AnalyticsCharts, BurnoutRadar, HabitHeatmap, QuickWaterLogger, DailyLogModal, HistoryManager, AiAdvisorModal, EducationalHub, ReportModal, SettingsModal, AdminConsoleModal).	Renders the interface, validates input and shows results.
State	App.jsx state (account, profile, logs, selected date, theme, modals).	Holds the session view, calls the account module and triggers saves.
Business logic	scoring.js, aiService.js, storage.js.	Scores, risk heuristic, recommendations, advisor, CSV / JSON helpers.
Account service	localAccount.js.	Registration, sign-in, session pointer, saving, administrator functions.
Browser APIs	IndexedDB, localStorage, Web Crypto (PBKDF2, getRandomValues).	Persistent storage, session pointer, hashing and random tokens.
Data	IndexedDB database hnazer-browser-data (object store accounts).	Persistent storage of accounts, profiles and logs.

CHAPTER 4: MODULE DESCRIPTION AND PROCESS LOGIC

4.1 Module Description

The project is divided into nine modules. The scoring engine, dashboard, persistence and administration modules need the most effort because they combine calculations, charts, validation and storage transactions.

Module	Description	Effort
M1 Navigation, Theme and Settings	Header with brand, date selector (previous, next, Today), theme toggle and sign-out; Settings modal (name, MRN, targets, screen ceiling, Gemini key, purge logs).	Low
M2 Authentication and Session	Account screen (create account / sign in), PBKDF2 hashing, session pointer, session restore and sign-out (AccountGate, localAccount.js).	Medium
M3 Daily Journal Entry	Daily entry form, one record per date, edit and delete, quick +250 ml, water logger.	Medium

Module	Description	Effort
M4 Scoring and Analysis Engine	Metric scores, weighted wellness score, burnout heuristic, alerts and recommendations (scoring.js).	High
M5 Dashboard and Visualisation	Daily summary, vitality overview, metric cards, trend and mood charts, six-axis radar, 35-day heatmap (inline SVG).	High
M6 History, Backup and Reports	History search and filter, CSV export, JSON backup and restore, print report.	Medium
M7 AI Advisor and Reference Library	Built-in advisor, optional Gemini advisor, Protocols reference library.	Medium
M8 Browser Persistence Layer	IndexedDB open and upgrade, account add/get/put, queued saves, default administrator seeding, input validation and error mapping (localAccount.js).	High
M9 Administration Console	Admin home and console: account list and search, statistics, dossier, reset logs, delete account, full backup, change password.	High

Outputs and reports

Hnazer produces the following outputs for the signed-in user and the administrator.

Output	Input	Result
Vitality overview	Selected log and recent records	Wellness score, burnout summary, recommendations and record counts.
Metric cards	Selected date's log	Sleep, exercise, water, screen, mood and diet summaries with quick +250 ml.
Trend charts	Saved records	Lines for sleep, screen time and water over the last 7, 14, 30 records or all records, plus mood distribution.
Six-axis radar	Metric scores	Polygon of the six scores.
35-day heatmap	Per-record scores	Colour-coded grid; selecting a day changes the dashboard date.
History table	Saved records	Searchable, filterable list with edit and delete.
CSV export	Logs	Columns user_id, name, date, sleep_hour, exercise_time_hours, water_intake_litres, screen_time_hours, mood, diet, notes.
JSON backup	Profile and logs	File with app name, version, export time, profile and logs.
Printable report	Averages, triage, alerts, up to 10 recent records	Print-formatted summary; the browser can save it as PDF.
Advisor output	Profile and recent records	Built-in or Gemini analysis text.
Admin account list and statistics	All account records	Per-account counts and averages, totals, estimated data size, cross-account averages.
Admin full backup	All account records	JSON file with every account, profile and log (without password salts or hashes).

Tentative contents of the project report

1. Introduction and objectives; tools and environment used.
2. Analysis document: SRS, E-R diagram, data flow diagrams, class diagram, data dictionary.
3. Design document: modularisation, database design and integrity constraints, procedural design, user-interface design.
4. Program code with comments and description.
5. Testing: test-case design and outcome reports for unit, integration and system testing; debugging and code improvement.
6. Input and output screens.
7. Implementation of security.
8. Limitations, future scope and bibliography.

4.2 Process Logic

The logic of each module is given below. Figure 4.1 shows the overall workflow of the system.

M2 Authentication

1. Receive name, username and password; the form checks the rules (HTML constraints) and shows errors on the account screen.
2. The account module validates again (username pattern, name length, password length) and, for registration, checks that the username is new; for sign-in it looks the account up by the username index.
3. Hash the password with PBKDF2-SHA-256 (210,000 iterations) and a random salt (registration) or compare it with the stored hash (sign-in). An unknown username is hashed with a placeholder salt so that the failure looks the same.
4. Create a random 256-bit session token, store it with the account id in local storage and return the account, profile and logs.
5. The client loads the dashboard (or admin home); on later visits the session is restored from the two keys; sign-out removes them.

M3 Daily Journal Entry

1. Open the entry form with the selected date and enter sleep, exercise, water, screen time, mood, diet and notes within the form ranges.
2. Calculate the wellness score and save the record under its date, replacing any earlier record for that date.
3. Refresh dashboard, charts, heatmap and history; show confetti if the score is 80 or more.
4. Quick hydration adds 0.25 L to the selected date (creating a default log if needed); records can be edited or deleted from the history.

M4 Scoring and Analysis Engine

The calculations are fixed heuristics implemented in one module; they are not a clinically validated model. Each component score is between 0 and 100.

Metric	Scoring rule
Sleep	7–9 h: 100; 6 to under 7 h: 80; over 9 to 10 h: 85; 5 to under 6 h: 60; 4 to under 5 h: 35; under 4 h: 15; over 10 h: 65.
Water	water_intake / target_water × 100, rounded and capped at 100.
Exercise	exercise_time / target_exercise × 100, rounded and capped at 100.
Screen	Up to 3.5 h: 100; up to the configured maximum: 85; up to maximum + 2 h: 60; up to maximum + 4 h: 35; higher: 15.
Mood	Energized 100; Good 85; Neutral 70; Tired 45; Stressed 35; Burnout Alert 15.
Diet	Healthy 100; Balanced 85; Average 65; Fast Food 40; Irregular 30.

Weighted composite: sleep 25% + exercise 20% + screen 20% + water 15% + mood 10% + diet 10%, rounded and limited to 0–100. The sleep target affects the progress display only; the sleep score itself uses the fixed bands above.

Vitality status. The composite is labelled “Optimal Vitality” (85 or more), “Healthy & Balanced” (70–84), “Mild Fatigue” (55–69) or “High Burnout Risk” (below 55).

Recent burnout heuristic. Up to the seven most recent saved records are used (not necessarily seven consecutive dates). Points are added for each condition met:

Condition over recent records	Points
Average sleep below 6 hours	30
Average screen time above 7 hours	25
Average water below 2 litres	15
Average exercise below 0.3 hours (18 minutes)	20
At least 3 records marked Tired, Stressed or Burnout Alert	20

The total is capped at 100: under 25 points is Low, 25–49 is Moderate and 50 or more is Critical. Text alerts and recommendations are created for the triggered conditions; an average sleep above 9.5 hours adds an informational alert without points. With no logs the engine returns “No Data” and a prompt to create an entry.

M5 Dashboard and Visualisation

1. Load the profile and logs after sign-in; show the daily summary banner (user name, ID, risk status, current sleep, water and score) and the vitality overview.
2. Show six metric cards for the selected date, with the +250 ml hydration action.
3. Draw trend charts for sleep, screen time and water over the last 7, 14, 30 saved records or all records, plus the mood distribution (the water line is scaled to share the axis with hour-based lines, as stated in the legend).
4. Draw the six-axis radar from the metric scores and the 35-day heatmap; selecting a heatmap day changes the dashboard date.

M6 History, Backup and Reports

1. List records with their score; search by date or notes and filter by mood; edit or delete a record (deletion asks for confirmation).
2. Export CSV (logs only) or download a JSON backup (app name, version, time, profile and logs).
3. Restore JSON replaces the account's whole log array; the profile and targets are not restored from the file.
4. Print Report opens a summary with averages, triage, alerts, recommendations and up to 10 recent records; the browser's print dialog can save it as PDF.

M7 AI Advisor and Reference Library

1. The built-in advisor derives a recent-record summary and a fixed action plan using local rule-based analysis; it uses no machine-learning model and sends nothing out.
2. The Gemini advisor runs only when explicitly chosen: the browser sends the API key and a prompt (profile name, targets and up to seven recent records without notes) to Google's Generative Language API and shows the structured reply (summary, risk, findings, action plan, encouragement).
3. The Protocols library shows reference text on sleep, screen time and eye care (20-20-20 rule), hydration and stress relief.

M8 Browser Persistence Layer

1. On start, the module opens the database, creates the store and index if needed and seeds the default administrator account if no "admin" exists.
2. getAccountSession reads the two session keys, loads the account and returns the account, profile and logs (an empty array for the administrator); missing accounts clear the keys.
3. Whenever the account, profile or logs change in App.jsx, saveAccountData chains a task on a promise queue that reads the current record and writes it back with the new profile and logs. Administrator accounts are never written this way.
4. New accounts are added with add(); in the success handler the generated id is copied into profile.user_id and the record is written again inside the same transaction.

M9 Administration Console

1. The administrator signs in on the same account screen and sees the admin home instead of the dashboard.
2. Every administrator function first calls requireCurrentAdmin, which reads the stored account and rejects it unless its role is "admin".
3. The Accounts tab lists all accounts with counts and averages and allows search, inspection of a read-only dossier, resetting logs and deleting an account (after confirmation; the signed-in administrator cannot be deleted).
4. The Statistics tab shows account and log totals, estimated data size, a session indicator and cross-account averages; the Tools tab downloads a full JSON backup and changes the administrator password after verifying the current one.

Overall workflow

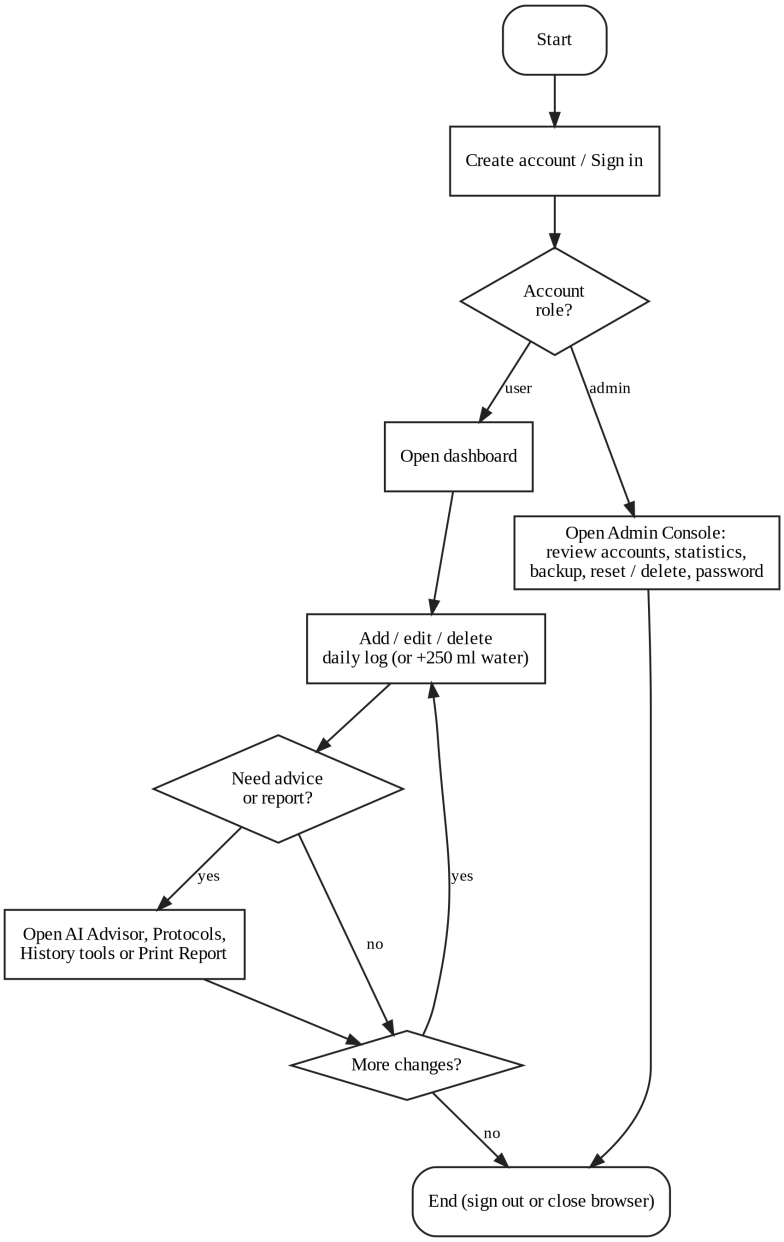


Figure 4.1: Overall workflow of the system

Walkthrough. A user starts by creating an account or signing in. The dashboard opens and the user adds, edits or deletes daily logs; each change immediately updates the metric cards, charts, radar, heatmap and history, and is saved automatically. An administrator who signs in instead is taken to the admin home and opens the Admin Console.

Decision 1 – Need advice or report? If the user wants advice, the AI Advisor, Protocols, history tools or Print Report can be opened from the dashboard at any point; they only read the saved data, so the user can close them and continue.

Decision 2 – More changes? If more entries are needed, the flow returns to the daily log; otherwise the session ends, either by signing out or by closing the browser. The session pointer stays in local storage until the user signs out.

CHAPTER 5: ANALYSIS

5.1 Use Case Diagram

A visitor can create an account or sign in. A registered user adds and reviews daily logs, views the dashboard, scores and burnout risk, uses the advisor and reference library, exports or restores data, prints the report, changes settings and purges logs. The administrator signs in with the same screen and then reviews accounts, statistics and dossiers, resets or deletes accounts, downloads a full backup and changes the administrator password. The Gemini API takes part only when the optional Gemini advisor is chosen.

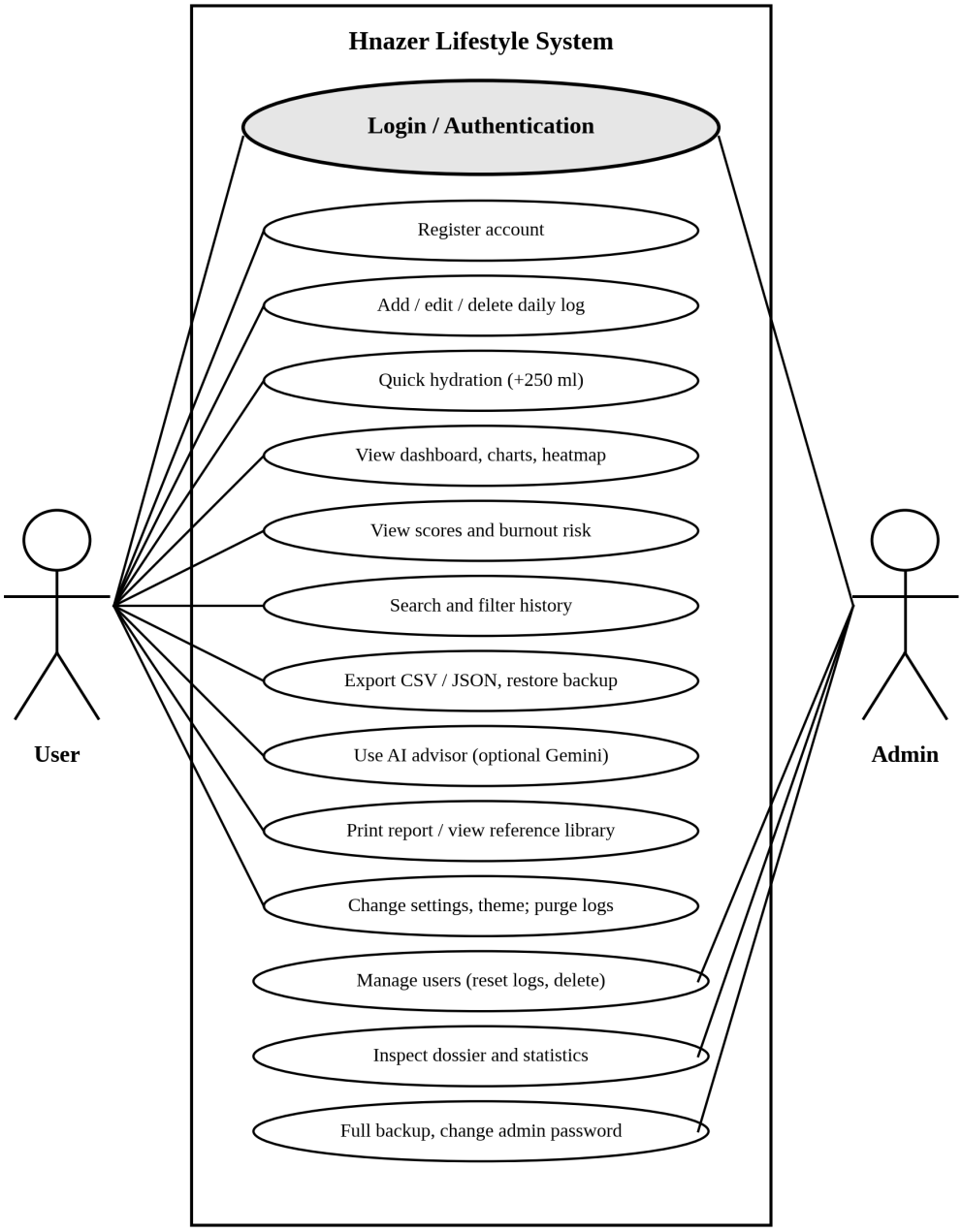


Figure 5.1: Use case diagram

Actors. Four actors take part. The *Visitor* has not signed in and can only reach the account screen. The *Registered user* has the private features that work on personal habit data. The *Administrator* has the account-maintenance features and no personal logbook. The *Gemini API* is an external system that only receives a prompt when the user explicitly runs the Gemini advisor.

Relationships. Registration and sign-in are the entry points: a successful sign-in opens the dashboard (user) or the admin home (administrator). There is no password-recovery use case in this version because the system has no e-mail

address to send a code to.

Pre- and post-conditions.

- *Manage daily log:* the user must be signed in; afterwards the dashboard, charts, heatmap and history are refreshed and the account record is saved.
- *View burnout summary:* at least one saved record should exist, otherwise “No Data” and a prompt are shown; the result is calculated fresh from saved records.
- *Use Gemini advisor:* a Gemini API key has been entered and the user explicitly chooses Gemini; the prompt holds the profile name, targets and up to seven recent records without notes.
- *Restore JSON:* the file is a log array or an object with a logs array; the account’s whole log array is replaced, so a backup should be taken first.
- *Purge logs / Reset logs:* the user (or administrator) confirms; the account, password and profile settings remain.
- *Delete user:* the administrator confirms; the whole account record is removed and the administrator cannot delete the account that is signed in.

Boundary. The system boundary encloses all use cases; everything outside it (visitors, users, administrator and the Gemini API) is an external actor. The table below summarises the main use cases.

Use case	Actor	Description
Create account / Sign in	Visitor	Creates an account or signs in; a session pointer is stored and the dashboard (or admin home) opens.
Add / edit / delete daily log	Registered user	Stores one entry per date with its wellness score; edits and deletes later.
Quick hydration	Registered user	Adds 250 ml to the selected date, creating a default log if needed.
View dashboard and scores	Registered user	Shows metric cards, charts, radar, heatmap, wellness score and burnout risk.
Use AI advisor	Registered user, Gemini API	Shows built-in advice, or Gemini advice if explicitly chosen.
Export / back up / restore	Registered user	Downloads CSV or JSON and restores logs from JSON.
Print report	Registered user	Opens a print-formatted summary that the browser can save as PDF.
Review and maintain accounts	Administrator	Lists accounts and statistics, inspects a dossier, resets logs, deletes an account, downloads a full backup and changes the administrator password.

5.2 Class Diagram

The class diagram shows the main data classes and the modules that use them. The App component holds client state; LocalAccountService (localAccount.js) handles accounts, persistence and administrator functions; ScoringService and AiService hold the calculation and advisor logic; StorageUtil holds the export helpers. Operation names are descriptive; the corresponding code is in App.jsx, localAccount.js, scoring.js, aiService.js and storage.js.

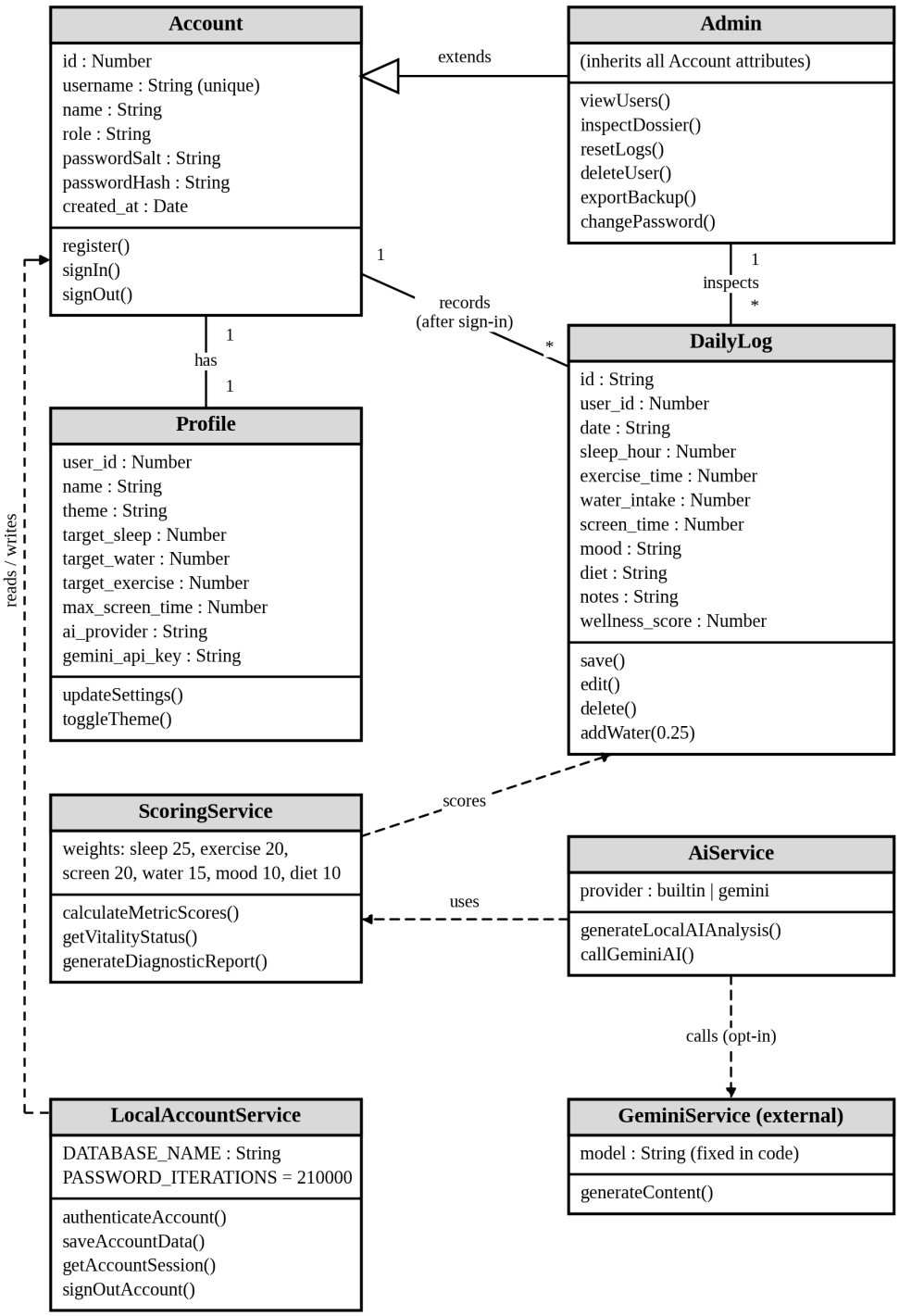


Figure 5.2: Class diagram

5.3 Data Flow Diagram

The data flow diagrams use the Yourdon notation: rectangles are external entities, ellipses are processes and open-ended boxes are data stores. The context diagram is decomposed into a Level 1 diagram and a Level 2 diagram that opens up the main processes. The data stores are D1 Accounts (account rows in IndexedDB), D2 Account Data (the profile and logs embedded in each account record) and D3 Session (the session pointer in localStorage).

6.3.1 Context diagram (Level 0)

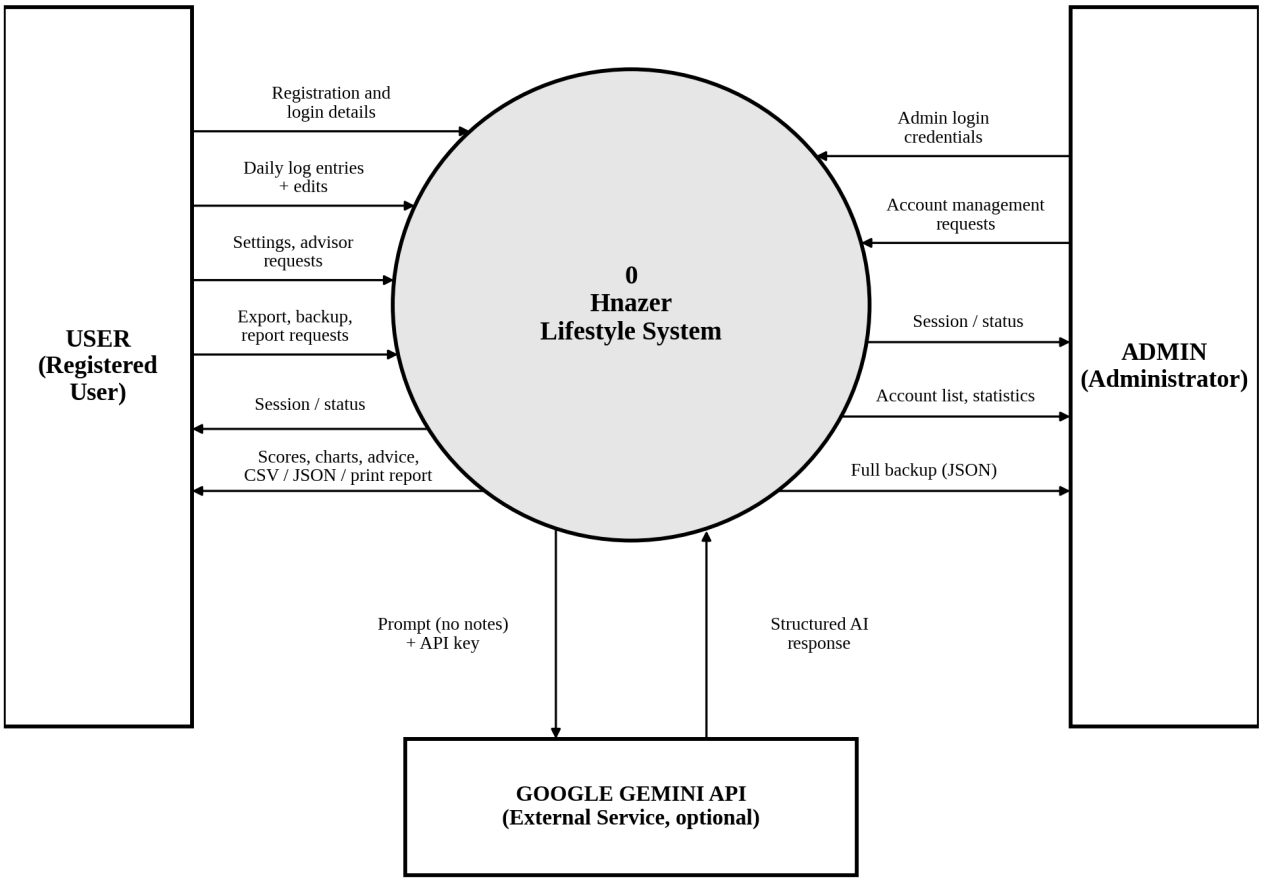


Figure 5.3: Level 0 DFD (context diagram)

The whole system is a single process. The user supplies registration, login, daily-entry, settings and advisor requests and receives authentication results, scores, charts, advice and CSV, JSON and report outputs. The administrator sends account-maintenance requests and receives lists, statistics and backups. The system reads and writes records in the browser database and, only on explicit choice, exchanges a prompt and a response with the Gemini API.

6.3.2 Level 1 DFD

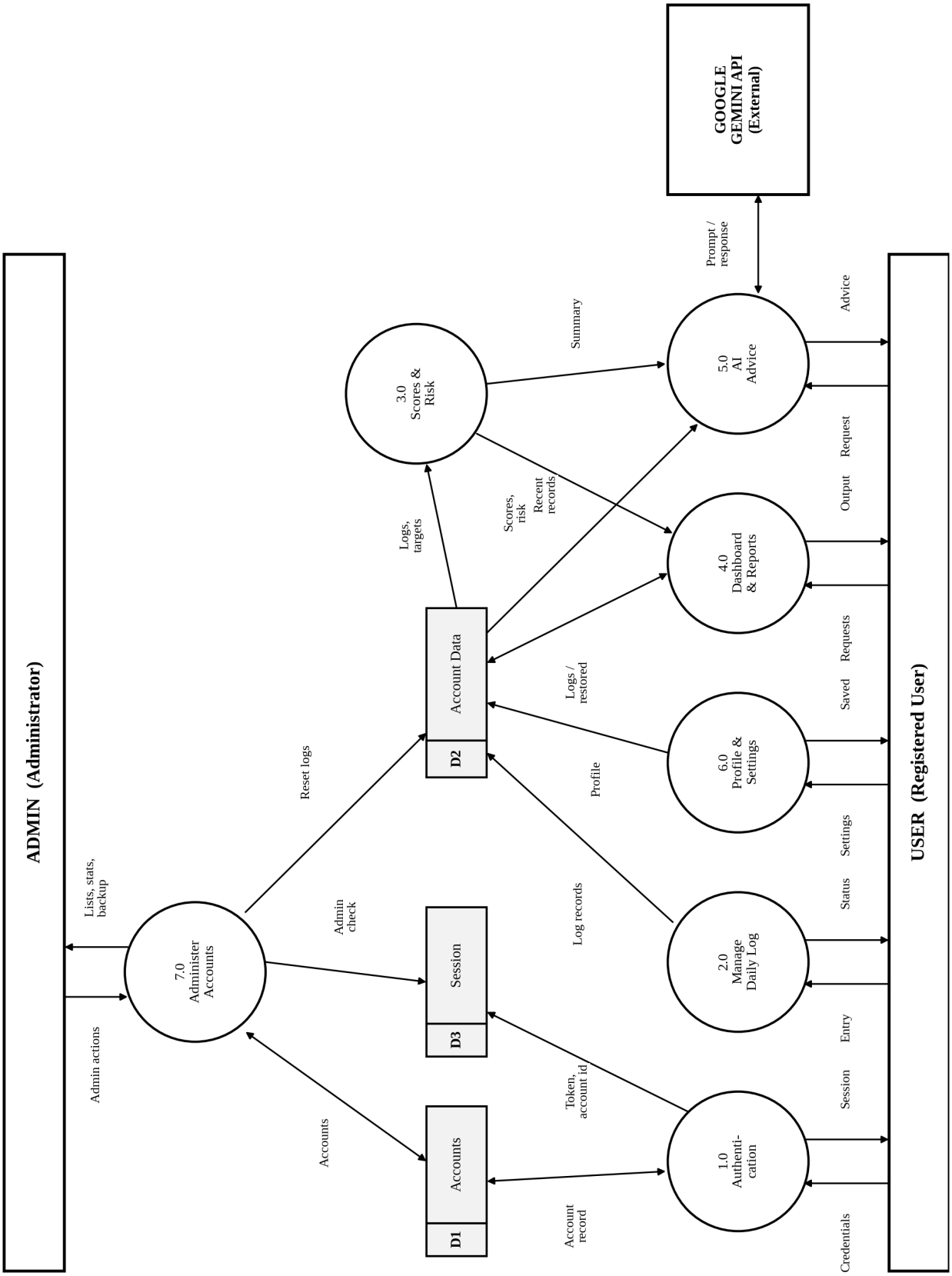


Figure 5.4: Level 1 DFD

Process	Description
1.0 Authenticate User	Validates credentials, reads or writes D1 Accounts, hashes or verifies the password and creates or removes the session pointer in D3 Session.
2.0 Manage Daily Log	Validates an entry, calculates its wellness score and adds, replaces, edits or deletes a record in D2 Account Data; includes quick hydration.
3.0 Calculate Scores and Risk	Reads logs and targets from D2 and computes metric scores, the composite score, burnout risk, alerts and recommendations.
4.0 Generate Dashboard and Reports	Builds cards, charts, radar, heatmap and history; produces CSV, JSON backup and the printable report; restores logs from JSON.
5.0 Provide AI Advice	Runs the built-in advisor or, if chosen, builds a prompt and calls the Gemini API.
6.0 Manage Profile and Settings	Updates display name, targets, theme and API key in the profile; purges logs.
7.0 Administer Accounts	Checks the administrator role, lists accounts and statistics, shows dossiers, resets logs, deletes accounts, exports the full backup and changes the administrator password.

6.3.3 Level 2 DFD

The Level 2 diagram opens up the main processes of the Level 1 diagram in a single figure: authentication (1.0), daily log (2.0), scores and risk (3.0), AI advice (5.0) and administration (7.0).

- **Process 1.0 – Authenticate User.** Input is validated, the account is found or created (the default administrator is seeded on first use), the password is hashed (registration) or verified (sign-in) with PBKDF2, and a session pointer is written to local storage.
- **Process 2.0 – Manage Daily Log.** Every entry is validated, scored and stored for its date; edits, deletions and quick hydration update the same record list, which is saved to the account record through the save queue.
- **Process 3.0 – Calculate Scores and Risk.** Records and targets are read, six metric scores and the weighted composite are calculated, and the most recent records give the burnout-risk level, alerts and recommendations.
- **Process 5.0 – Provide AI Advice.** The profile and recent records are read; the built-in analysis runs locally, or, only if chosen, a prompt without notes is sent to Gemini and the reply is shown.
- **Process 7.0 – Administer Accounts.** The administrator role is checked, accounts are listed and inspected, logs are reset, accounts are deleted, the password is changed and a full backup is produced.

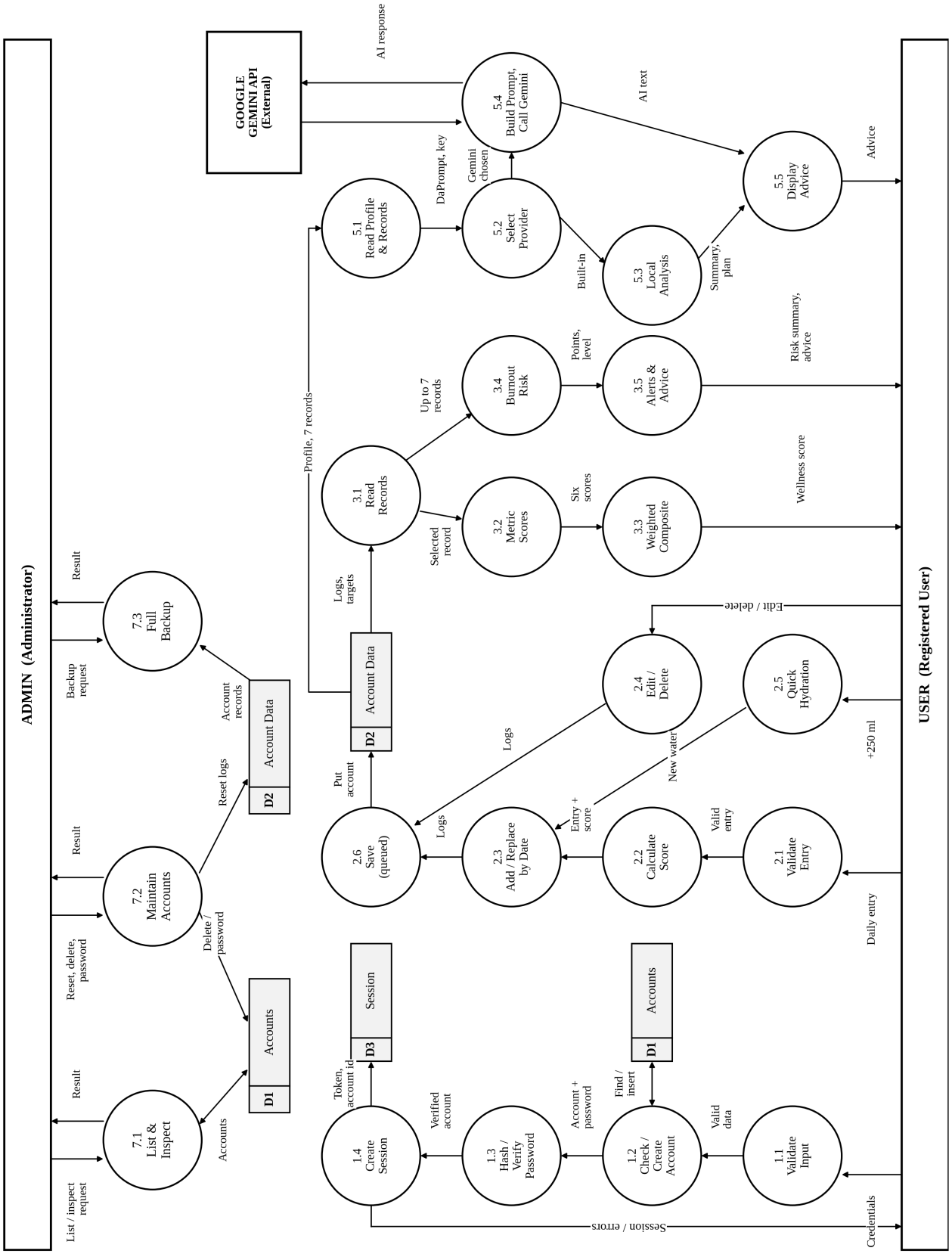


Figure 5.5: Level 2 DFD (processes 1.0, 2.0, 3.0, 5.0 and 7.0)

5.4 Entity Relationship Diagram

The browser database has one object store. Each record in the accounts store is one account and carries its profile and its daily logs as embedded structures, so the profile and the daily log are logical entities rather than separate stores. The session pointer is a separate pair of localStorage keys that refers to one account.

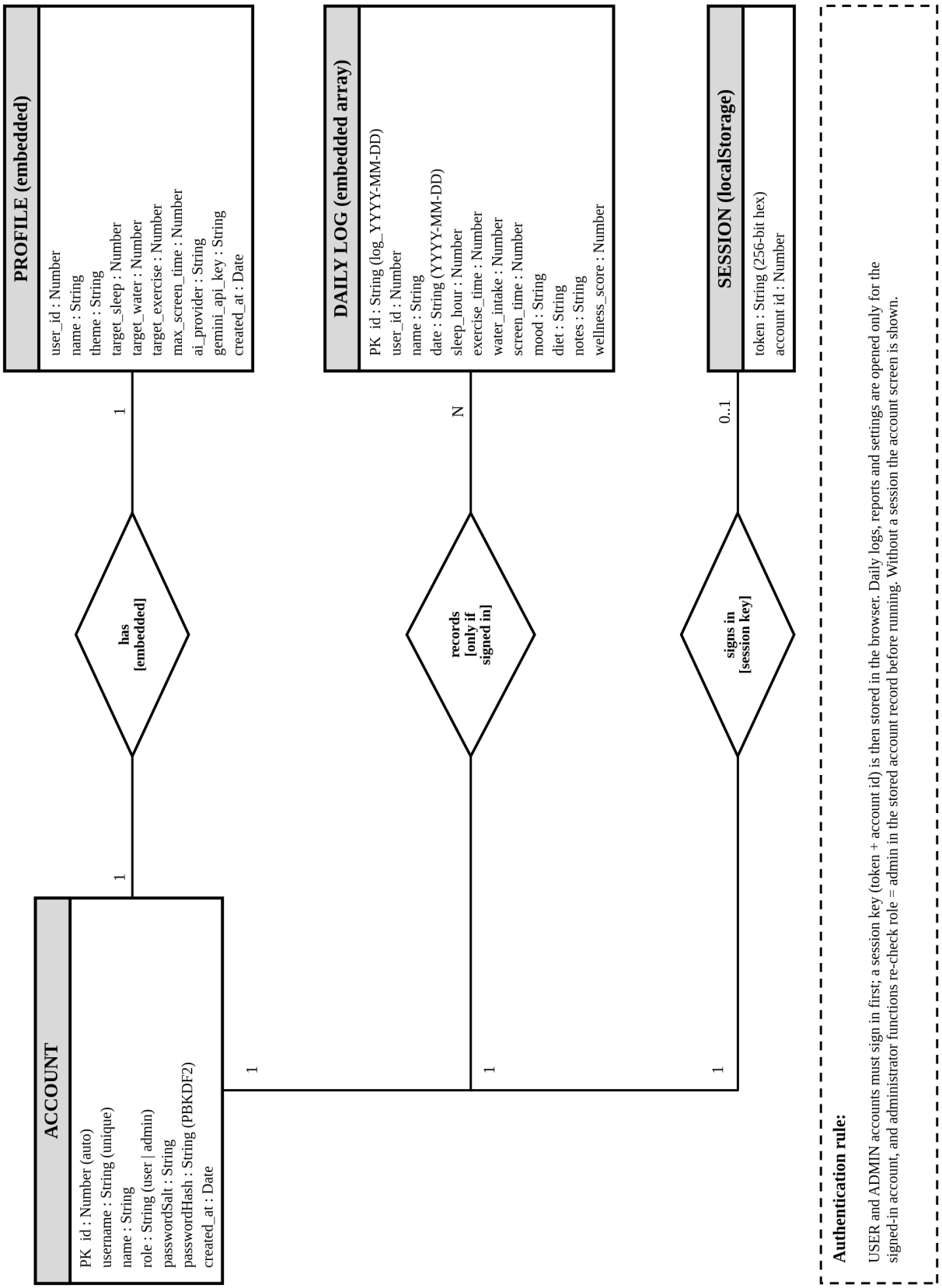


Figure 5.6: Entity-Relationship diagram

Entities and keys. ACCOUNTS is the main entity; its id is the auto-increment key and its username is covered by a unique index. The profile is embedded (1 : 1) and the daily logs are embedded as an array (1 : N). The SESSION pointer stores a random token and the id of the signed-in account.

Relationships and cardinality.

- **has (logical):** one account holds exactly one profile (1 : 1); a new account receives it at registration and its user_id equals the account id.
- **contains (logical):** one account holds many daily logs (1 : N) as an array; there is one log per date. Administrator accounts always hold an empty array.
- **current sign-in:** a browser holds at most one session pointer at a time (1 : 0..1); signing in elsewhere replaces it.

Referential actions. Because profile and logs are embedded, deleting an account record removes its profile and logs with it; no orphan rows are possible.

Integrity rules. Usernames are unique, 3–32 characters; passwords are stored only as salt and hash; the add operation maps a unique-index violation to the message “That username already exists in this browser.”; user_id is set to the new id inside the same transaction as the insert; all saves go through one queue so that two writes never overlap.

Design note. Keeping the logbook inside the account record keeps the prototype simple: one read and one write move the whole logbook. The trade-off is that individual days cannot be queried or indexed; a later version may move logs into their own object store (Section 9.2).

Database design

The database is created by the browser the first time the application opens it: an IndexedDB database named hnazer-browser-data, version 1, with one object store. It lives in the user’s browser profile for the site origin; different browsers, profiles, devices and origins have separate data. The schema is applied automatically in the upgrade handler.

Integrity and constraints

- The accounts store uses an auto-increment key path (id); the unique index on username rejects duplicates, and the error is shown as a readable message.
- Passwords are stored only as salt and PBKDF2 hash. Profile and logs are embedded in the same record, so they are deleted automatically with the account.
- Writes use read-write transactions; the insert and the user_id update for a new account happen in one transaction.
- The signed-in session is only a pair of localStorage keys; it is removed on sign-out or when the referenced account no longer exists.
- The database contents are not encrypted at rest; protection relies on the operating-system account and the browser profile.

5.5 Data Structure

Structure	Kind	Purpose
Account record	{ id, username, name, role, passwordSalt, passwordHash, created_at, profile, logs }	Account identity, profile and logbook stored in IndexedDB.
Profile object	{ user_id, name, target_sleep, target_water, target_exercise, max_screen_time, theme, ai_provider, gemini_api_key, created_at }	Personal settings and targets.
Daily log	{ id, user_id, name, date, sleep_hour, exercise_time, water_intake, screen_time, mood, diet, notes, wellness_score }	One day's journal entry.
Metric scores	{ compositeScore, sleepScore, waterScore, exerciseScore, screenScore, moodScore, dietScore }	Input to the composite score and radar.
Risk result	{ burnoutIndex, riskLevel, riskColor, averages, alerts, recommendations }	Burnout summary; "No Data" when no logs exist.
Session pointer	Two localStorage strings (token, account id)	Identifies the signed-in account after a reload.
Save queue	A chained promise (saveQueue)	Makes sure saves run one after another.
Chart datasets	Arrays of dates and values (last 7, 14, 30 records or all)	Feed the trend and mood charts.
Heatmap grid	Last 35 dates with per-record scores	Colour-coded activity overview.
Export formats	CSV columns user_id, name, date, sleep_hour, exercise_time_hours, water_intake_litres, screen_time_hours, mood, diet, notes; JSON backup with app name, version, time, profile and logs	Portable copies of the data.
Admin summaries	User rows { id, username, name, role, created_at, totalLogs, avgSleep, avgWater, avgScreen, latestDate }; system statistics	Feed the Admin Console tabs.
React state objects	account, profile, logs[], selectedDate, open modals	Application state kept in the App component.

Data dictionary

The data dictionary lists the attributes of the four entities shown in the ER diagram. ACCOUNT is stored as a record in the accounts object store; PROFILE and DAILY LOG are embedded inside that record; SESSION is kept in browser local storage.

ACCOUNT (IndexedDB object store "accounts", keyPath id, auto-increment; unique index on username)

Field	Type	Constraints and description
id	Number	Primary key (auto-increment).
username	String	Login identity; trimmed, lower-case, 3–32 letters, digits or underscores; unique index.
name	String	Name shown in the app; required, up to 80 characters.
role	String	"user" for registered users, "admin" for the administrator.
passwordSalt	String	Random 16-byte salt (32 hexadecimal characters) used for the PBKDF2 hash.
passwordHash	String	PBKDF2-SHA-256 hash (210,000 iterations, 256 bits) of the 8–128 character password; never the plain password.
created_at	Date	ISO creation timestamp.

PROFILE (embedded in the account record)

Field	Type	Default and description
user_id	Number	Account id; MRN / user identifier shown in the app.
name	String	Registration name; display name.
theme	String	dark by default; light or dark theme.
target_sleep	Number	8 hours; sleep target (UI range 4–12 h, step 0.5); affects the progress display only.
target_water	Number	3 litres; water target (1–8 L, step 0.25).

Field	Type	Default and description
target_exercise	Number	0.75 hours; exercise target (0.25–4 h, step 0.25).
max_screen_time	Number	5 hours; screen-time ceiling (2–14 h, step 0.5).
ai_provider	String	builtin by default; chosen advisor provider.
gemini_api_key	String	Empty by default; optional Gemini API key (stored as plain text).
created_at	Date	Profile creation date (YYYY-MM-DD).

DAILY LOG (embedded array in the account record; one entry per date)

Field	Type	Description
id	String	Primary key of the entry, generally log_YYYY-MM-DD.
user_id	Number	Profile / MRN identifier.
name	String	Profile display name copied into the record when saved.
date	String	YYYY-MM-DD; used to find or replace the day's entry.
sleep_hour	Number	Sleep duration in hours (form range 0–14, step 0.5).
exercise_time	Number	Exercise in hours (0–3, step 0.25).
water_intake	Number	Water in litres (0–6, step 0.25).
screen_time	Number	Screen exposure in hours (0–16, step 0.5).
mood	String	Energized, Good, Neutral, Tired, Stressed or Burnout Alert.
diet	String	Healthy, Balanced, Average, Fast Food or Irregular.
notes	String	Optional free text.
wellness_score	Number	Composite score (0–100) calculated when the entry is saved.

SESSION (browser local storage)

Field	Type	Storage key and description
token	String	hnazer_browser_session_v1: random 32 bytes as 64 hexadecimal characters; its presence marks a signed-in browser.
account id	Number	hnazer_browser_account_v1: id of the signed-in account, used to load the account record when the page is reopened.

CHAPTER 6: SYSTEM REQUIREMENT SPECIFICATION AND FEASIBILITY STUDY

6.1 Software

Purpose and overview

This SRS describes the functions and constraints of Hnazer. The intended readers are the project guide, the evaluator and the developer. The system has two internal user classes (the registered user and the administrator), plus visitors and one optional external service (Google's Gemini API). It runs in a modern web browser that supports IndexedDB and Web Crypto. It assumes that users enter truthful figures and understand that scores and advice are informational heuristics, not medical diagnoses.

Functional requirements

ID	Module	Requirement
FR-01	Authentication	Create an account with a name, username and password.
FR-02	Authentication	Validate registration: username trimmed, lower-cased, 3–32 letters, digits or underscores and unique in the browser; name required (up to 80 characters); password of 8–128 characters.
FR-03	Authentication	Store passwords only as salted PBKDF2-SHA-256 hashes (210,000 iterations); never in plain text.
FR-04	Authentication	Sign in with username and password; on success create a random session token and keep it, with the account id, in browser local storage. Show one generic message for any failed sign-in.
FR-05	Authentication	Restore the session on later visits; sign-out removes the session keys and returns to the account screen.
FR-06	Authentication	Create a default profile (with user_id equal to the account id) and an empty log array at registration.
FR-07	Daily entry	Record date, sleep (0–14 h), exercise (0–3 h), water (0–6 L), screen time (0–16 h), mood (six categories), diet (five categories) and optional notes.
FR-08	Daily entry	Keep one log per date: saving the same date replaces the earlier entry; entries can be edited and deleted (deletion asks for confirmation).
FR-09	Daily entry	Calculate and store the wellness score on save; show a confetti animation at a score of 80 or more.
FR-10	Daily entry	Add 250 ml quickly from the hydration card or adjust the day’s water total; create a default log if the date has none.
FR-11	Analysis	Calculate six metric scores (0–100) from fixed rules and the user’s targets.
FR-12	Analysis	Combine the metric scores into a weighted composite (sleep 25%, exercise 20%, screen 20%, water 15%, mood 10%, diet 10%).
FR-13	Analysis	Calculate a burnout-risk level (Low, Moderate, Critical) from up to the seven most recent records, with alerts and recommendations; return “No Data” when there are no logs.
FR-14	Dashboard	Show the daily summary banner with status, the vitality overview and six metric cards for the selected date.
FR-15	Dashboard	Show trend charts (7, 14, 30 days or all records) for sleep, screen time and water, plus mood distribution.
FR-16	Dashboard	Show a six-axis radar and a 35-day heatmap whose cells select the dashboard date.
FR-17	History	List records with score; search by date or notes; filter by mood; edit and delete.
FR-18	History	Export logs as CSV and the profile and logs as a JSON backup.
FR-19	History	Restore a JSON file (raw array or object with a logs array), replacing the account’s logs.
FR-20	AI advisor	Provide a built-in advisor using local rule-based analysis that sends no data out.
FR-21	AI advisor	Provide an optional Gemini advisor, used only on explicit choice, sending the key and a prompt without the notes field.
FR-22	Reference	Provide an in-app reference library (Protocols) with four topics: sleep, screen time and eye care, hydration, and stress relief.
FR-23	Report	Provide a print-formatted report with averages, triage summary, alerts, recommendations and up to 10 recent records.
FR-24	Settings	Edit display name, MRN / user ID, daily targets, screen-time ceiling and Gemini API key.
FR-25	Settings	Purge the current account’s logs after confirmation; keep the account and profile.
FR-26	Settings	Provide a light/dark theme switch in the header and save the choice in the profile.
FR-27	Dashboard	Provide a date selector with previous-day, next-day and Today actions.
FR-28	Persistence	Automatically save the profile and logs of a signed-in user account to IndexedDB after every change, through a serial save queue.
FR-29	Administration	Seed a default administrator account (role “admin”) when the database has none; administrator accounts hold no daily logs.
FR-30	Administration	List all accounts with role, creation date, log count, average sleep, water and screen time and latest log date; search the list.
FR-31	Administration	Open a read-only dossier (account, profile and logs) of any account.

ID	Module	Requirement
FR-32	Administration	Reset the logs of a user or delete a user account, each after confirmation; an administrator cannot delete the account currently signed in.
FR-33	Administration	Show system statistics: number of accounts and logs, estimated browser data size, session indicator and cross-account averages.
FR-34	Administration	Download a full JSON backup of all accounts (without password salts or hashes).
FR-35	Administration	Change the administrator password after verifying the current one (new password 8–128 characters).
FR-36	Administration	Check the administrator role from the stored account record before every administrator function.

Non-functional requirements

Attribute	Requirement
Security	Passwords hashed with salted PBKDF2-SHA-256; session tokens random (256-bit); unknown usernames take the same hashing path as known ones and return the same message; every administrator function re-checks the role from the stored record.
Privacy	Data stays in the user's browser unless the user explicitly chooses Gemini; the Gemini prompt excludes the notes field; no e-mail address is collected; the developer cannot see visitors' data.
Usability	Single dashboard, simple forms, light and dark themes, responsive layout (compact controls and a floating add button on small screens).
Performance	A personal logbook is small; the whole account record is read in one operation and written in one operation, so pages respond almost instantly.
Reliability	Writes are serialised through one promise queue; IndexedDB transactions are atomic; storage errors are caught and reported instead of crashing the page; a missing account removes stale session keys.
Portability	Runs in current Chrome, Edge, Firefox or Safari on Windows, macOS, Linux or mobile. Web Crypto needs a secure context (HTTPS or localhost).
Maintainability	Separate components and utility modules; scoring rules kept in a single module; account storage kept in a single module.
Scalability	The prototype targets one browser profile. Each save rewrites the whole account record; moving to multi-device use would need a server, stronger authentication and a separate log table.

Tools and technologies

Layer	Technology	Purpose
Front end	React 18, Vite 6	User interface and development server / build tool.
Styling	Tailwind CSS 3, PostCSS, Autoprefixer	Responsive layout and light/dark themes.
Charts	Inline SVG written in React components	Trend and mood charts, radar and heatmap.
Icons and effects	lucide-react, canvas-confetti	Interface icons; celebration animation.
Security	Web Crypto API (PBKDF2, getRandomValues)	Password hashing, random salts and tokens.
Database	IndexedDB (browser)	Persistent storage of accounts, profiles and logs; no server installation.
Session	localStorage	Session token and account id.
AI	Built-in rule-based advisor; Google Gemini API (optional)	Local analysis; optional external AI analysis.
Utilities	npm; Google Fonts (remote)	Package management and fonts.

The package list also names chart.js and react-chartjs-2, but the current code draws all charts itself and does not import them.

Software requirements

- Operating system: Windows, macOS or Linux capable of running Node.js (for development); any modern operating system for using the built site.
- Runtime for development: a current Node.js LTS release with npm.

- Browser: a current version of Chrome, Edge, Firefox or Safari with JavaScript, IndexedDB, local storage and Web Crypto enabled (HTTPS or localhost is required for Web Crypto).
- AI: a Gemini API key (optional; without it only the built-in advisor is used).
- Commands: npm ci, npm run dev, npm run build (writes the static site to the sibling site folder), npm run preview.

6.2 Hardware

Profile	Recommended specification
Minimum (development)	Dual-core 2 GHz processor, 4 GB RAM, 2 GB free storage, internet connection.
Recommended (development)	Four-core processor, 8 GB RAM, SSD storage, broadband internet.
Client (runtime)	Any laptop, desktop, tablet or phone that can run a modern browser.

The specifications are estimates and depend on the amount of data.

6.3 Feasibility Study

Feasibility	Assessment
Technical	All technologies (React, Vite, Tailwind CSS, IndexedDB, Web Crypto) are mature, open source or built into modern browsers, and are well documented. No server runtime is needed in production.
Economic	Development needs only a computer and an internet connection to install packages. IndexedDB ships inside the browser, so no paid database or hosting server is needed; the built site is static and can be hosted free of charge. A Gemini API key is required only if the optional Gemini advisor is used.
Operational	The system runs in a normal web browser and starts with one command (npm run dev) or from the static build. A single dashboard, a simple daily form and a built-in advisor that needs no external service make it easy for non-technical users.

CHAPTER 7: SYSTEM DEVELOPMENT LIFE CYCLE

The iterative-incremental model is followed. The phases are requirement analysis, design (architecture, DFDs, ER diagram, data model, screens), implementation (React front end, IndexedDB account module, scoring engine, Gemini integration, administrator console), testing (Chapter 8) and deployment as a static build, with maintenance. An earlier prototype that used a local Node.js service with SQLite was replaced by browser-only storage so that no server is needed. Issues expected during development, such as IndexedDB transaction and unique-index errors, Web Crypto needing a secure context, and Gemini API errors or invalid keys, are handled with readable error messages and are treated as challenges to be re-tested, not as permanently resolved items. The planned schedule is 12 weeks (about 84 days), as shown in Figure 7.1.

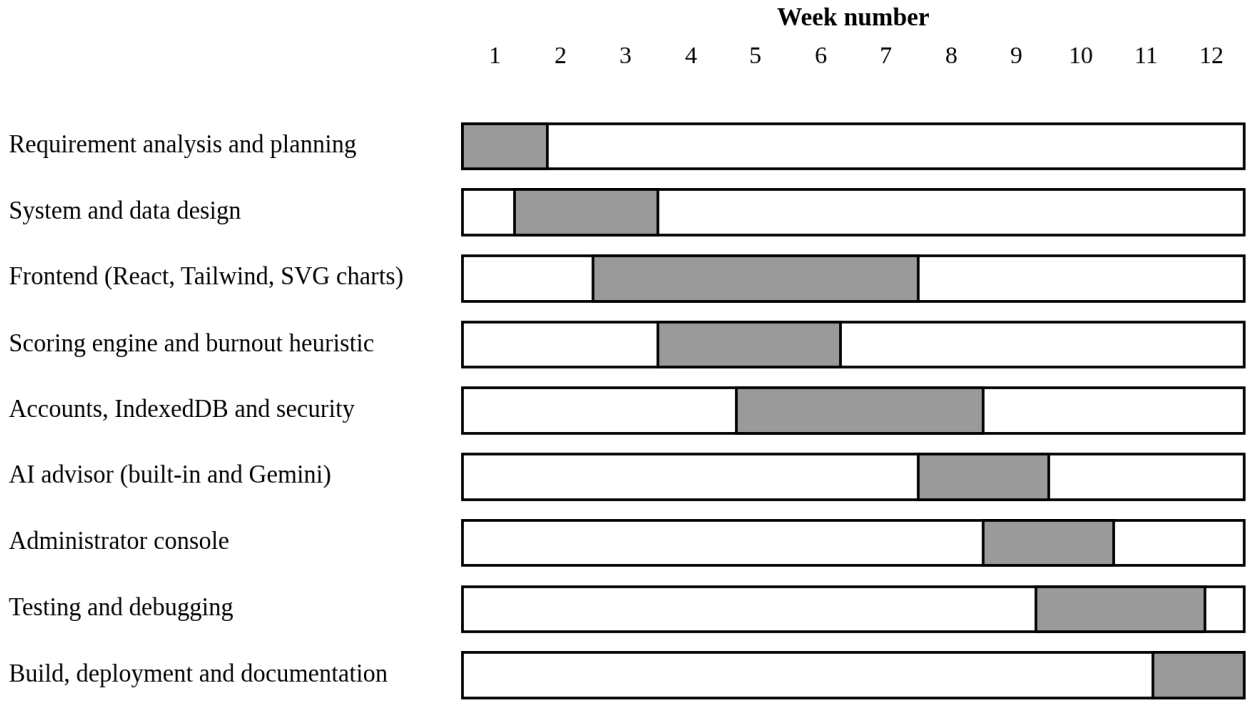


Figure 7.1: Project schedule (12 weeks)

CHAPTER 8: TESTING AND SECURITY

8.1 Testing of the Project

Testing follows the levels required by the guidelines. Test cases are designed for each level, results are recorded in the project report, and defects found are logged, debugged and corrected. The current package has no test or lint script, so the automated tests below are planned work; the production build (npm run build) is the present compile check. The expected values for the scoring and risk cases were confirmed by running the project's scoring.js with the stated inputs.

Level	Focus	Approach
Unit testing	Scoring rules, weighted composite, burnout heuristic, input validation, CSV and JSON helpers.	Vitest with sample inputs (planned).
Integration testing	Account module with IndexedDB: registration, sign-in, queued saves, administrator functions, session restore and sign-out.	Vitest with a fake IndexedDB (planned).
System testing	Complete journeys in the browser: sign-up, daily entry, dashboard, history, backup and restore, advisor, settings, administrator console.	Manual test scripts and browser automation.
Acceptance testing	Clarity of the dashboard, scores and advice with sample users.	Guided trials with test data.

Sample test cases

ID	Module	Test case	Expected result
TC-01	Auth	Create an account with a valid name, username and 8+ character password.	Account created; dashboard opens with an empty logbook.
TC-02	Auth	Register a username that already exists.	Message “That username already exists in this browser.”; no duplicate account.
TC-03	Auth	Use the username “ab”, or one with spaces or symbols.	Validation error; account not created.
TC-04	Auth	Use a password of 7 characters.	Password-length error; no account.
TC-05	Auth	Sign in with a wrong password.	Generic message “Username or password is incorrect.”; no session keys stored.
TC-06	Auth	Reload the page while the session keys are present.	Session restored; profile and logs load.
TC-07	Admin	Sign in as admin / admin1234 on a fresh browser.	Admin home shown; no daily-entry controls.
TC-08	Security	Call an administrator function without an administrator session.	Error “Sign in as an administrator to continue.” and no data returned.
TC-09	Security	Sign in as two accounts one after the other.	Each session loads only its own profile and logs.
TC-10	Entry	Save sleep 8 h, exercise 0.75 h, water 3 L, screen 3 h, Energized, Healthy (default targets).	Wellness score 100; confetti animation.
TC-11	Entry	Save a second entry for the same date.	Earlier entry replaced; one record for that date.
TC-12	Score	Save sleep of 5.5 h (also 3.9 h, 4 h and 10.5 h).	Sleep score 60 (and 15, 35 and 65).
TC-13	Score	Sleep 6.5 h, exercise 0.375 h, screen 6 h, water 1.5 L, Tired, Average (default targets).	Metric scores 80, 50, 60, 50, 45, 65 (sleep, exercise, screen, water, mood, diet); wellness score 61.
TC-14	Risk	Three records with sleep 5 h, screen 8 h, water 1.5 L, exercise 0.1 h and mood Stressed.	30 + 25 + 15 + 20 + 20 = 110, capped at 100; level Critical.
TC-15	Risk	Open the dashboard with no logs.	“No Data” and a prompt to create an entry.
TC-16	Hydration	Press +250 ml on a date with no log.	New record with default values and water 0.25 L.
TC-17	History	Edit one record and delete another (confirm); reload.	Changes persist in IndexedDB.
TC-18	Backup	Download JSON backup, purge logs, restore the file.	Logs return; profile unchanged.
TC-19	AI	Run the built-in advisor.	Local analysis shown; no Gemini request made.
TC-20	AI	Run the Gemini advisor with an invalid key.	Error shown; the application stays usable.
TC-21	Settings	Change the water target to 2 L and save (water 1.5 L).	Water score 75 (was 50 at 3 L); setting persists.
TC-22	Admin	Reset the logs of a user from the Admin Console.	That user’s logs are empty at next sign-in; account remains.
TC-23	Admin	Try to delete the signed-in administrator account.	Error “Cannot delete your own active administrator account.”
TC-24	Admin	Download the full backup.	JSON with all accounts, profiles and logs; no password salts or hashes.

8.2 Security Implementation

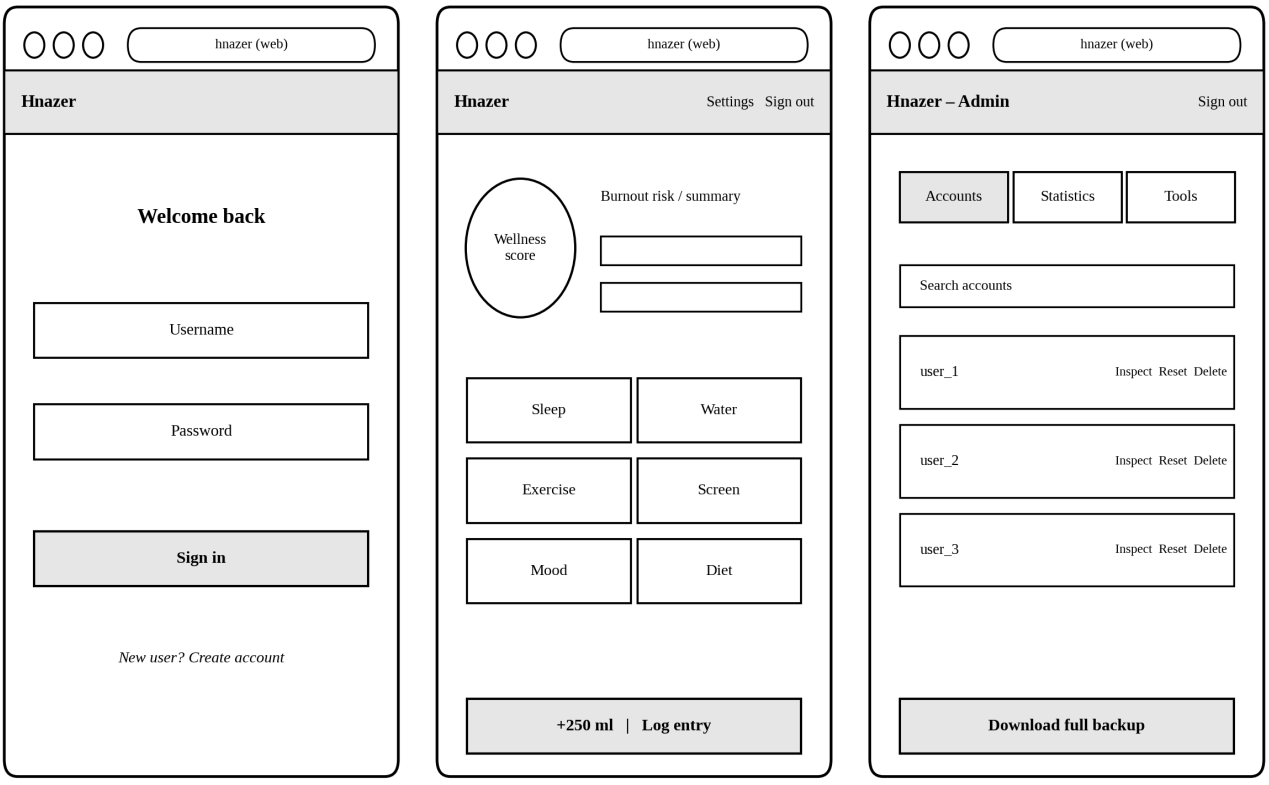
- Passwords are hashed with a random 16-byte salt using PBKDF2-SHA-256 (210,000 iterations, 256-bit output) through the Web Crypto API before storage; the plain password is never stored. Passwords longer than 128 characters are rejected.
- A failed sign-in always returns the same message; for an unknown username the same hashing work is done with a placeholder salt, so the response does not reveal which part was wrong.
- Session tokens are 256-bit random values from crypto.getRandomValues, stored with the account id in local storage; sign-out removes them. The token is a convenience pointer in the user’s own browser, not a server-verified credential.

- Administrator functions call requireCurrentAdmin, which loads the account from the database and checks the role; the administrator's own account cannot be deleted from the console, and the administrator password can be changed only after the current one is verified.
- A default administrator account (admin / admin1234) is created automatically; its password should be changed after the first sign-in.
- The built-in advisor sends nothing out; Gemini is opt-in, and its prompt omits the notes field.
- **Known gaps are stated openly:** data in IndexedDB and exports are not encrypted; anyone with access to the browser profile or developer tools can read or change local data, so the administrator role is not a security boundary against the device owner; the Gemini key is saved in the profile as plain text, appears in JSON backups (and in the administrator's full backup) and is sent as a URL parameter to Google; there is no rate limiting or lockout. The prototype is meant for personal use in a single browser profile.

CHAPTER 9: INDUSTRY CLIENT INTERFACE AND FUTURE SCOPE

9.1 Industry Client Interface

The client interface is a responsive web interface built with React and Tailwind CSS. The main screens are the sign-in / create-account page, the dashboard with daily entry, charts and advisor, and the administrator console, as shown in the proposed layouts of Figure 9.1. Final screenshots will be added from the working application. The project is an individual academic work and is not developed for any industry or client.



(a) Sign in / Create account **(b) Dashboard** **(c) Admin Console**

Figure 9.1: Proposed screen layouts (wireframes)

9.2 Future Scope and Further Enhancement

Future scope

- A server-backed version with proper authentication, password recovery with e-mail or OTP verification, account deletion and two-factor authentication.
- Encryption at rest (for example with a key derived from the password), hashed or externally held API keys and rate limiting for sign-in.
- A separate object store (or relational table) for daily logs, enabling indexed queries and long-term reports.

- Wearable and sensor integration for measured sleep, activity and heart-rate data.
- Editable score weights and personalised thresholds; validated models in place of fixed heuristics.
- Direct PDF report generation, CSV import, consecutive-day streaks and calendar-based chart ranges.
- Automated unit, integration and end-to-end tests with continuous integration.
- An installable progressive web app or mobile application with optional synchronisation.

References

1. R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, McGraw-Hill Education.
2. I. Sommerville, Software Engineering, Pearson Education.
3. R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, Pearson Education.
4. React Documentation – https://react.dev
5. Vite Documentation – https://vite.dev/guide/
6. Tailwind CSS Documentation – https://tailwindcss.com/docs
7. MDN Web Docs: IndexedDB API – https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API
8. MDN Web Docs: Web Crypto API (SubtleCrypto.deriveBits, PBKDF2) – https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API
9. MDN Web Docs: Window.localStorage – https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
10. Google Gemini API Documentation – https://ai.google.dev/gemini-api/docs
11. OWASP Cheat Sheet Series: Password Storage and Session Management – https://cheatsheetseries.owasp.org
12. IGNOU School of Computer and Information Sciences, BCSP-165 Project Guidelines.